Notes (Week 2 Wednesday)

These are the notes I wrote during the lecture.

```
Formal languages

Formal language are
  sets of words

Words are strings (sequences of symbols)
  over some *alphabet* Σ

  Σ is a finite set of symbols

Σ^* the set of all words over Σ

A *language* over Σ is a subset of Σ^*


Example:

  {hello,world}·{salut,monde}
  = {hellosalut,hellomonde,
     worldsalut,worldmonde}

If A is a language, A* consists of:
  any finite sequence of concatenated
  words from A

hello ∈ {hello,world}*
hellohello ∈ {hello,world}*
hellohelloworldhello ∈ {hello,world}*
λ ∈ {hello,world}*

DIG = {0,1,2,3,4,5,6,7,8,9}

D = {λ,-}·DIG·DIG*·({λ} ∪ {.}·(DIG ∪ DIG^2))


 {}^* = {λ}

 {λ}^* = {λ}


A relation is a subset of a Cartesian product.

  ≤  ⊆ ℕ × ℕ

  (x,y) ∈ ≤   iff x is less than or equal to y
```

```
  x ≤ y

  ≤ = {(x,y) : (x,y) ∈ ℕ × ℕ and
               there exists z ∈ ℕ and
                 x+z=y}

  (0,3) ∈ ≤
  ≡ (by definition)
  there exists z ∈ ℕ and 0+z=3
  ≡ (too obvious)
  True

Notation: For *binary* relations, we often write
 them in infix.

  (x,y) ∈ R    x R y   R(x,y)

 R(A), if R is a binary relation R ⊆ S × T,
 and A ⊆ S
 gives you every T that is related to some S.

 R = {(mango,tasty),(banana,yucky),
      (mango,sweet)}

 R({mango}) = {tasty,sweet}
 R({banana}) = {yucky}

 R({mango,banana}) = {tasty,yucky,sweet}
 R({}) = {}

 R^←  converse of a relation

 R^← = {(tasty,mango),(yucky,banana),
        (sweet,mango)}

 ≤^← = ≥

 If x = y and y = x then x = y

Def: If x R y and y R x then x = y

A not-symmetric relation,
 and an antisymmetric relation,
 is not the same thing.
```

	R	AR	S	AS	T
=	✓		✓	✓	✓
≤	✓			✓	✓
<		✓		✓	✓
∅	?	✓	✓	✓	✓
𝓊	✓	?	✓	?	✓

```
  ∅ ⊆ ℕ × ℕ
  ∅ ⊆ ∅ × ∅
Reflexive means
  ∀x ∈ S. (x,x) ∈ R



  If I'm a frog then I'm deep-fried

If a relation ~ is reflexive, symmetric and
  transitive, we call it an equivalence relations.

  ≡ (logical equivalence) is an equivalence





 R = {(x,y) | x,y ∈ ℕ and x mod 3 = y mod 3}

 R happens to be reflexive, symmetric and transitive

   [1] = {1,4,7,...}

Note: [1] = R({1})

One nice property of equivalence classes:
  x R y

  [x] = [y]

For elements x,y that are not equivalent

  [x] ≠ [y]

  [x] and [y] are disjoint




Partial orders:
  (R),(AS),(T)

Strict partial orders:
  (AR),(AS),(T)

 ≺ "precedes"
 ≼ "precedes or equals"
 ≻ "succedes"

In the slides, ≼ is just a variable name,
 and denotes some arbitrary partial order

  2|-2   and  -2|2  but 2 ≠ -2

Note: definitions in slides are for
```

```
  strict partial orders.
These are for partial orders.

Minimal element:
  an x such that for every y ≼ x, x = y
Minimum element:
  an x such that x ≼ y for y

First, an obvious(?) point:
 minimal and minimum elements do not always
 exist.

 Consider (ℤ,≤)



Functions map each input to exactly one
  output.

Functions are relations f ⊆ S × T
 between inputs and outputs.

 f(x) = y  means the same as (x,y) ∈ f

 not a function:
   R = {(1,2),(1,1)}
 is a function:
   g = {(1,1),(2,1)}

we might write g as:
   g : {1,2} → {1,2}
   g(x) = 1

   domain {1,2} (inputs)
   codomain {1,2} (outputs)
   image  {1}
   the image is always a subset of the
   codomain, but might not cover it

The input-output relation that characterises
a function f is sometimes called
the *graph* of f.

   g : ℝ → ℝ
   g(x) = 5/x
   ^that's a partial function, because it's
    undefined at 0

 f(x) = x + 1

 (f ∘ f)x = f^2(x) = f(f(x)) = x + 2

 "the identity function on S"
 Id_S  : S → S
```

```
 Id_S(x) = x

 the identity function is the same thing as
 the identity relation

If R1 and R2 are relations,
  R1; R2  "the composition of R1 and R2"
relates (x,y) if y can be reached from
x with one hop through R1 and one hop through
 R2

 R1; R2 = {(x,y) | exists z. x R1 z and z R2 y}

 R = {(x,y) | x+1 = y}

 x R y means y is one larger x

 x (R;R) y  there must be some z
   such that z is one larger than x
    and  y is one larger than x
 in other words:
   y is two larger than x
```